

Lab 1: Fork-Join Model, Team Creation, and Thread Scoping

Language: Java 17

Experiment run: 24 September 2026

Program: ForkJoinLab1.java

1. Objective

The experiment creates a fixed-size ForkJoinPool, submits parallel work items, and waits for them to finish. It compares the time to create and join pools of different sizes, observes that task execution order can vary, and measures CPU use while each task performs a CPU-bound square-root loop.

2. System and software

Item	Recorded value
Computer	MacBook Air, Mac15,13
Processor	Apple M3, 8 cores (4 performance + 4 efficiency)
Logical processors reported by macOS	8
Memory	8 GB unified LPDDR5
L1 data / instruction cache	64 KiB / 128 KiB (system-reported)
L2 cache	4 MiB (system-reported)
L3 cache	Not reported by the queried macOS system properties
OS	macOS 15.2 (24C101)
Runtime	OpenJDK 17.0.18, Homebrew build

3. Method

The source was compiled with `javac ForkJoinLab1.java`. For the timing sweep, the program used `System.nanoTime()`, two warm-up runs per team size, then five measured runs for each $P = 1, 2, 4, 8, 16, 32, 64$. `workIterations=0` isolates pool/task overhead as much as this simple experiment allows. The ten-run log was generated by starting the program ten times. For the CPU observation, the program was run with 2,000,000 square-root-loop iterations per task and sampled with macOS `top` once per second.

The timing includes construction and shutdown of a new pool, task submission, task completion, and result aggregation. This is a small-operation benchmark, so timer noise and JVM/runtime overhead are a substantial part of the result.

4. Results

Elapsed time in milliseconds (five trials; mean calculated from those trials):

P	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Mean
1	0.088	0.101	0.089	0.080	0.082	0.088

P	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Mean
2	0.107	0.096	0.096	0.119	0.105	0.105
4	0.144	0.127	0.125	0.135	0.128	0.132
8	0.245	0.578	0.217	0.204	0.237	0.296
16	0.387	0.304	0.266	0.219	0.240	0.283
32	0.430	0.420	0.429	0.418	0.579	0.455
64	0.702	1.371	0.562	0.535	0.688	0.772

The ten-run log showed three distinct printed rank orders. The captured CPU samples recorded a Java process peak of approximately 506% CPU during the short load run. macOS process CPU is summed across cores. The raw samples are in lab1-top-samples.txt.

5. Reproduction and attached raw files

Compile and run from this folder:

```
bash
```

```
javac ForkJoinLab1.java
```

```
java ForkJoinLab1 0 > lab1-results.csv 2> lab1-threads.txt
```

```
java ForkJoinLab1 2000000 > lab1-load-results.csv 2> lab1-load-observation.txt
```

The ten-run observation and top sample are also included. Files in this folder: source, README, this report, timing CSV, ten-run output, worker output, load-run outputs, and CPU samples.